

eShop Performance Demo — real k6 runs

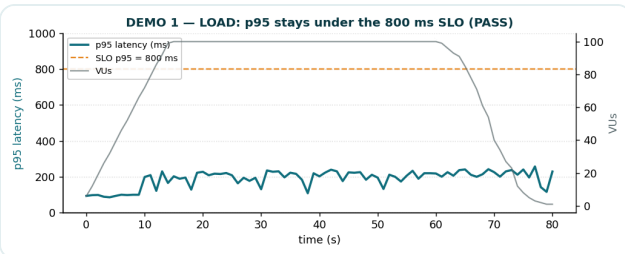
Three tests from the in-class deck, executed against the mock eShop. Every number and chart below is captured output — reproduce it with `bash demo/run-demo.sh`.

DEMO 1 · LOAD

Meet targets at peak

PASS · exit 0

Full shopping journey · ramp to 100 VUs, hold, ramp down · SLO p95 < 800 ms



checks (functional)	100.00% (4744/4744)
p95 latency	214.8 ms
checkout p95	254.7 ms
error rate	0.00%
throughput	7.4 iters/s
SLO verdict	ALL MET → exit 0

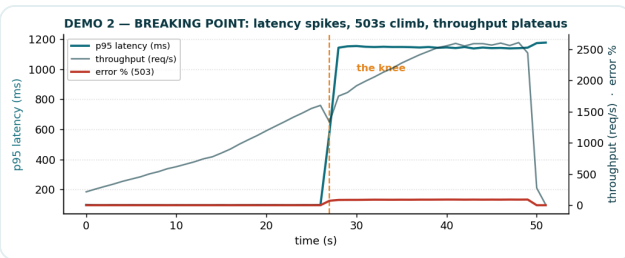
Read it: p95 sits flat near 200 ms — comfortably under the 800 ms SLO — the whole time VUs hold at 100. A green load test is the baseline everything else compares to.

DEMO 2 · BREAKING POINT

Push it until it sheds

BREACH · exit 99

Catalog read path · arrival rate ramped 200 → 2600 req/s



p95 latency	1.13 s (was 215 ms)
error rate (503)	62.2% (46907/75432)
throughput ceiling	~1475 req/s
dropped_iterations	523
checks 'not 503'	37.8%
SLO verdict	THRESHOLD CROSSED → exit 99

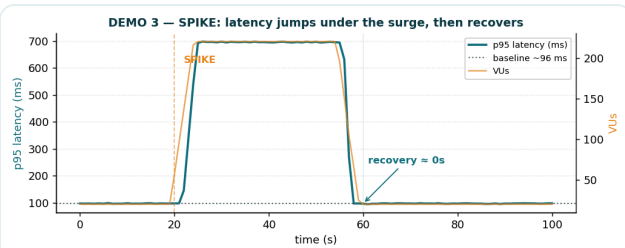
The knee: throughput climbs, then plateaus ~1475 req/s while p95 shoots from 215 ms to 1.13 s and 503s appear. The system **sheds load gracefully** (503, not a crash). That plateau is the capacity number.

DEMO 3 · SPIKE

The flash crowd, then recovery

RECOVERS

Catalog read path · jump 20 → 220 VUs in 5 s, sustain, drop back



baseline p95	~96 ms
peak p95 (under surge)	~698 ms
error rate	0.00%
checks	100.00% (34808/34808)
recovery time	≈ 0–1 s
SLO verdict	held → exit 0

Read it: latency jumps ~7× the instant VUs spike, holds during the surge, then snaps back to baseline within one measurement bucket when load drops. Recovery is near-instant here because the mock has **no queue backlog** — a real system with queues would show a longer recovery tail. That contrast is the teaching point.

REPRODUCE

Run the whole demo yourself

```
# 1) install k6 · 2) from the repo root:
bash demo/run-demo.sh          # runs all 3 tests, writes demo/out/
python3 demo/analyze.py        # rebuilds these charts from the JSON
```

```
# or one at a time:  
node mock/server.js &  
k6 run demo/load.js           # PASS (exit 0)  
k6 run demo/breakpoint.js     # BREACH (exit 99) – the knee  
k6 run demo/spike.js          # jumps, then recovers
```

Numbers here are from the mock eShop (hard limit 300 concurrent). A real eShop will break at different numbers — point the suite at it with **-e BASE_URL=http://localhost:<port>** and re-run.